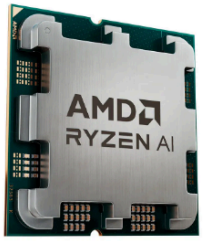
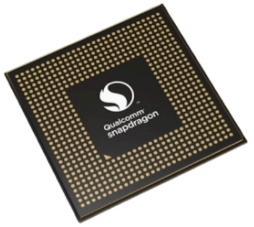
Software

Introduction to computer programming

Management and Artificial Intelligence

How can a computer execute a program?



The CPU is the responsible for executing the program code:

- It interprets the instructions one by one from the **RAM**
- The instructions must be written in **machine code**
- The machine code is:
    - **Low level**: quite simple operations
    - **CPU specific**: different CPUs have different machine codes
    - **Binary format**: humans struggle to read it

Machine code: Intel/AMD vs ARM

A simple sequence of instructions to compute the sum of two integer numbers would look like this in machine code:

Intel/AMD 64 bit machine code	ARM 64-bit machine code

`f3 0f 1e fa`
`55`
`48 89 e5`
`89 7d fc`
`89 75 f8`
`8b 55 fc`
`8b 45 f8`
`01 d0`
`5d`
`c3`
	`ff 43 00 d1`
	`e0 0f 00 b9`
	`e8 0f 40 b9`
	`e9 0f 40 b9`
	`00 7d 09 1b`
	`ff 43 00 91`
	`c0 03 5f d6`

- Each instruction is stored into its own line for clarity, but in practice, they would be stored contiguously in memory.
- The machine code is stored in a binary format, while in this example we have used hexadecimal notation for clarity.
- Intel and AMD processors use the same machine code format (with only minor differences).
- Other processors (e.g., ARM used by Apple, Raspberry Pi, Huawei, etc.) use a completely different machine code format.

From machine code to assembly code

To make machine code understandble for human, we can translate it to assembly code:

- **Low level** [same as machine code]: quite simple operations
- **CPU specific** [same as machine code]: different CPUs have different machine codes
- **Human readable** [differently from machine code]: CPU struggles to read it!

Intel/AMD 64 bit assembly

```
endbr64
push   %rbp
mov    %rsp,%rbp
mov    %edi,-0x4(%rbp)
mov    %esi,-0x8(%rbp)
mov    -0x4(%rbp),%edx
mov    -0x8(%rbp),%eax
add    %edx,%eax
pop    %rbp
ret
```

ARM 64-bit assembly

```
sub     sp, sp, #16
str     w0, [sp, #12]
ldr     w8, [sp, #12]
ldr     w9, [sp, #12]
mul     w0, w8, w9
add     sp, sp, #16
ret
```

Without getting into details, we can see that this code is doing some basic operations, such as addition, multiplication, copy/move of values from/to registers (which are different from CPU to CPU), etc.

NOTE: Translation from/to machine code to/from assembly code is a one-to-one non-ambiguous mapping. Translating from machine code to assembly code is called **disassembly**, while the opposite is called **assembly**.

Writing low level code is impractical

Major problems when working with machine/assembly code are:

1. Programmers have to write a lot of code to perform simple operations
2. Code is not portable across different hardware platforms
3. Programmers have to deal with hardware details
4. Programming in machine/assembly code is hard and error-prone

Yet, CPUs only understand and run machine code.

How can programmers survive?

From low-level to high-level programming

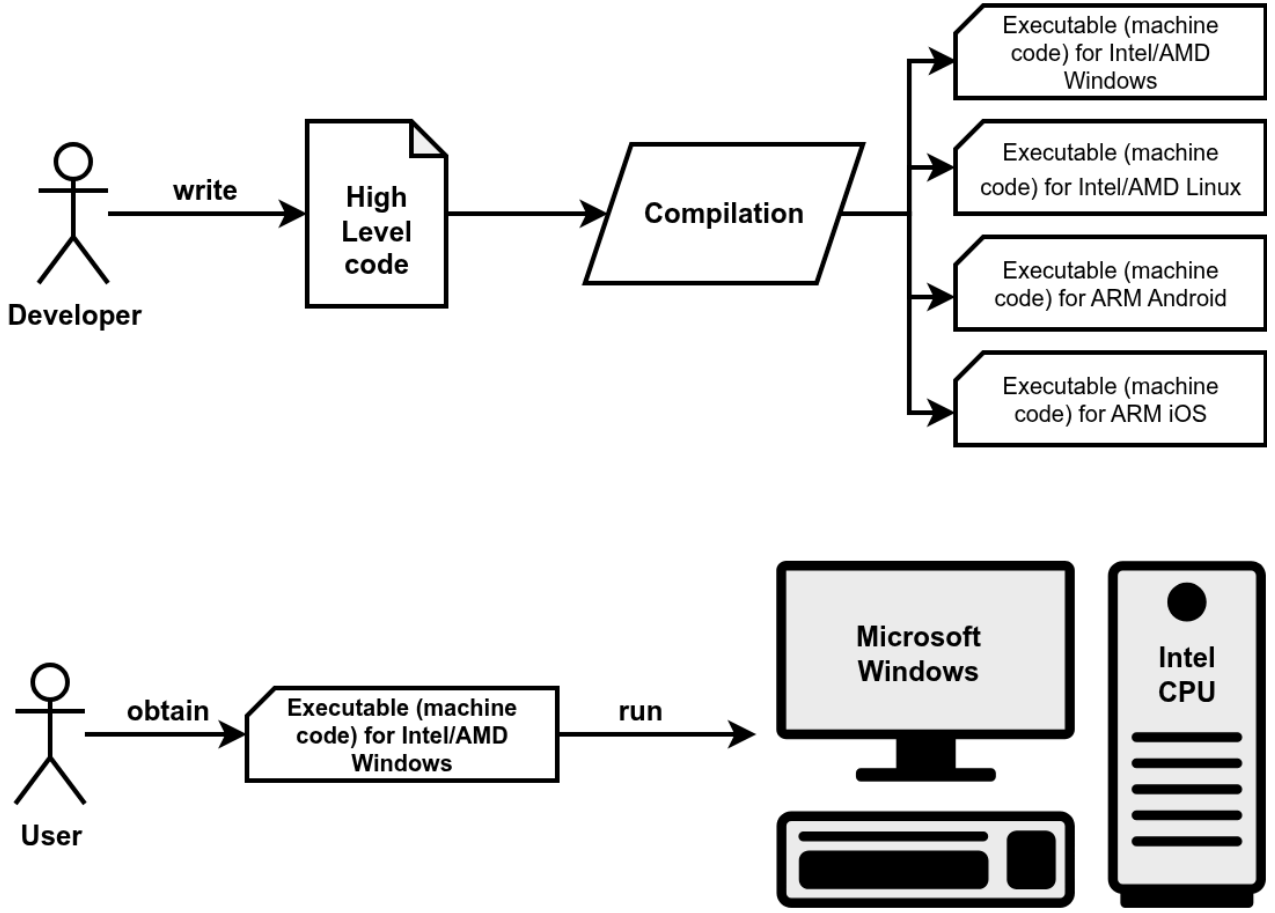
Most programmers use high-level languages, such as Python, Javascript, or PHP, to write software.

However, the CPU does not understand high-level languages. Never!

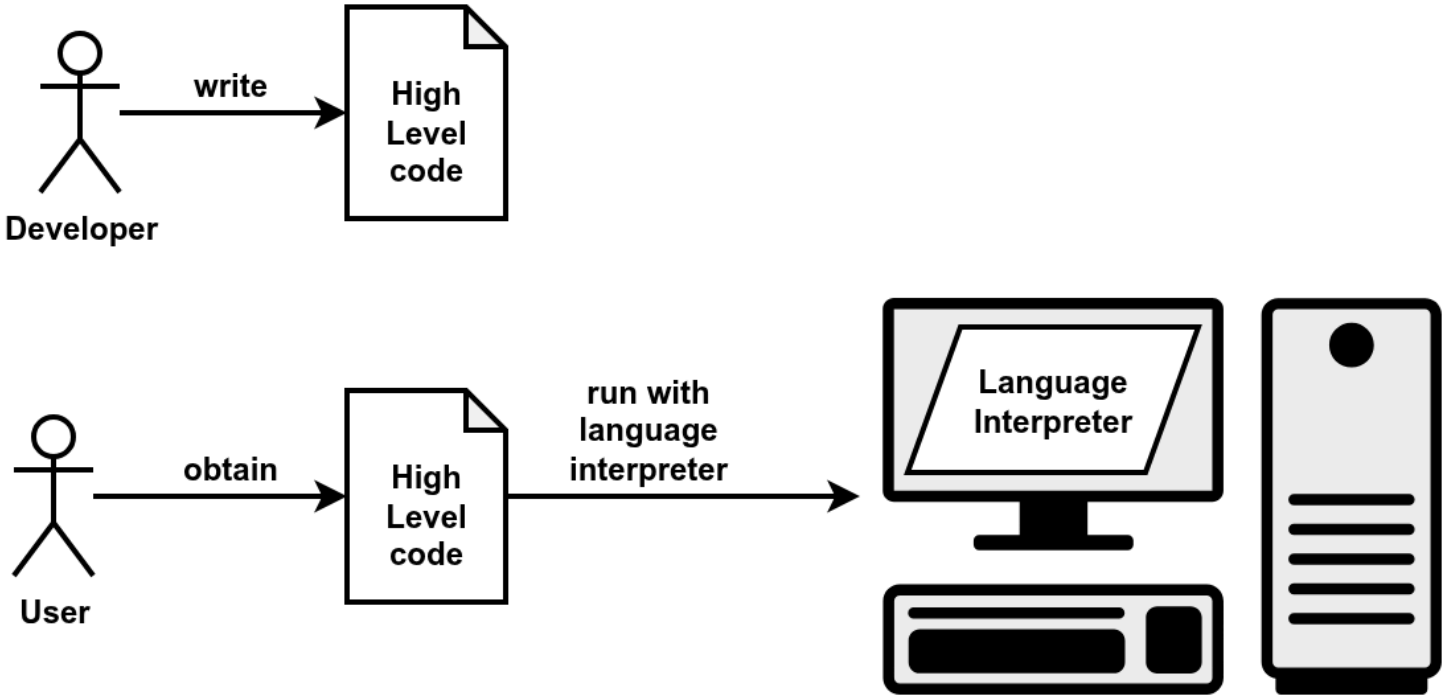
Code written in high-level languages must be translated into machine code. Two main strategies:

1. **Compilation**:
    - Compilers translate high-level code into machine code at *compile time*
    - Compile time means when the developer writes the code.
    - Machine code is packaged into an executable.
    - One executable for each platform (Operating Systems and CPUs).
    - Developers keeps the high-level code, while users get the low-level code.
2. **Interpretation**:
    - Interpreters translate high-level code into machine code at *running time*.
    - Running time means when the user runs the code.
    - The code is portable and platform independent.
    - Developers distribute the high-level code, users exploits the interpreter to run the code.

Compilation



Interpretation



Language: syntax and semantics

Programming languages have:

- **Syntax**: it defines which strings of characters and symbols are well formed.
- **Semantics**: it defines the meaning of the well formed strings.

The same is true even for natural languages (English, Italian, etc.):

- **Syntax**:
  - *"I are big"* is syntactically incorrect (*"I am big"* is correct).
  - *"The square circle is big"* is a well formed sentence.
- **Semantics**:
  - *"The square circle is big"* is semantically incorrect (nonsensical because "square" and "circle" contradict).

An important difference between programming languages and natural languages is that programming languages are designed to be **unambiguous**, which means that there is no room for interpretation.

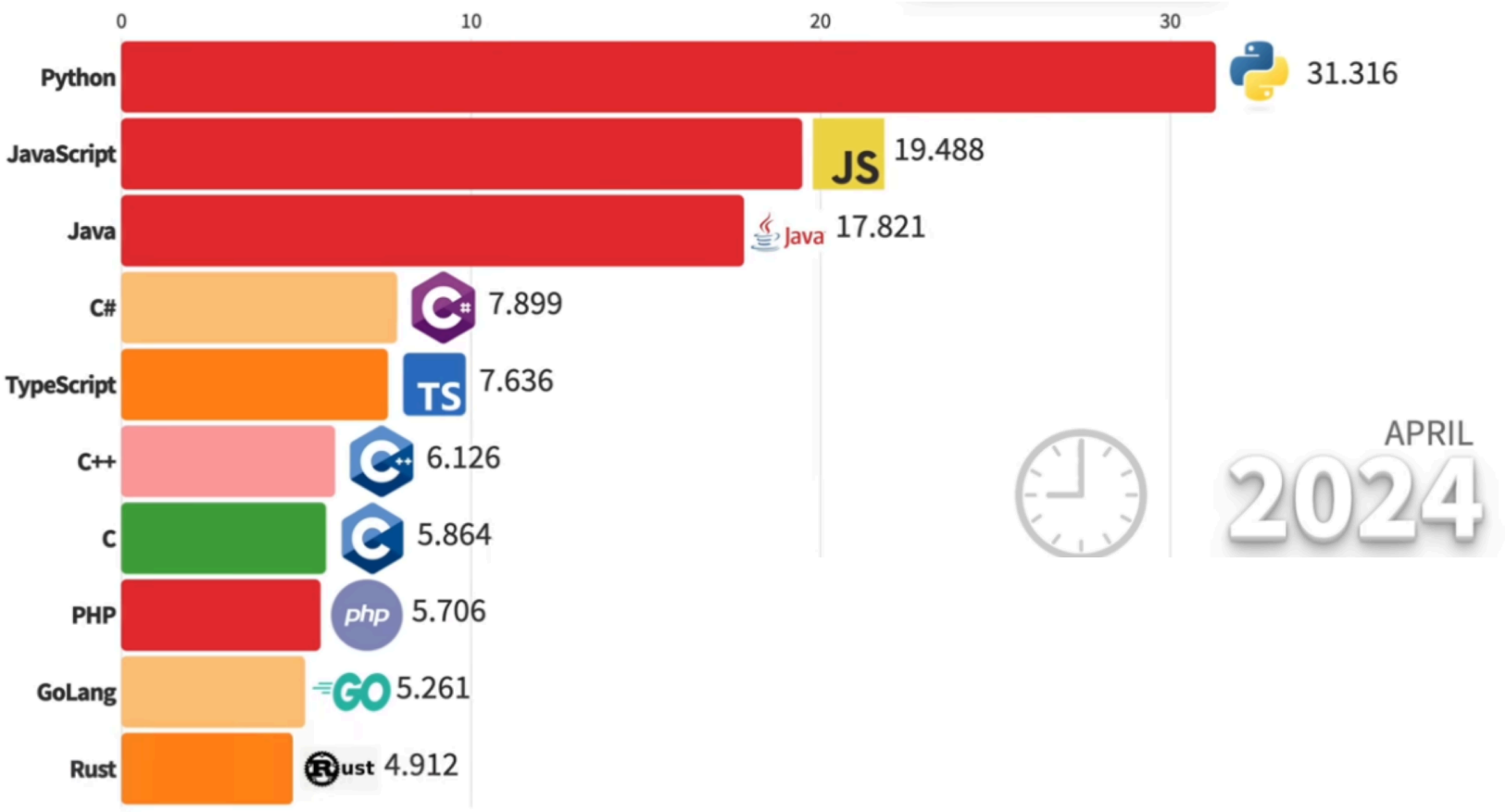
Different high-level languages

There exists thousands of high-level languages:

- They have been designed with different goals in mind
- Some are general purpose, some are domain specific (e.g., web)
- Some gives more control to the user, some hide more details
- Some are compiled, some are interpreted

In this course, we will focus on a single high-level language: ***but which we should choose?***

Popular programming languages



See the full video

Python

Algorithms

Operating Systems